ACU
AUSTRALIAN CATHOLIC UNIVERSITY

Software Processes

The Software Process

- A structured set of activities required to develop a software system
- Many different software processes but all involve:
  - Specification – defining what the system should do
  - Design and implementation – defining the organization of the system and implementing the system
  - Validation – checking that it does what the customer wants
  - Evolution – changing the system in response to changing customer needs
- A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective

Plan-driven and Agile processes

Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan

Agile processes, planning is incremental and it is easier to change the process to reflect changing customer requirements

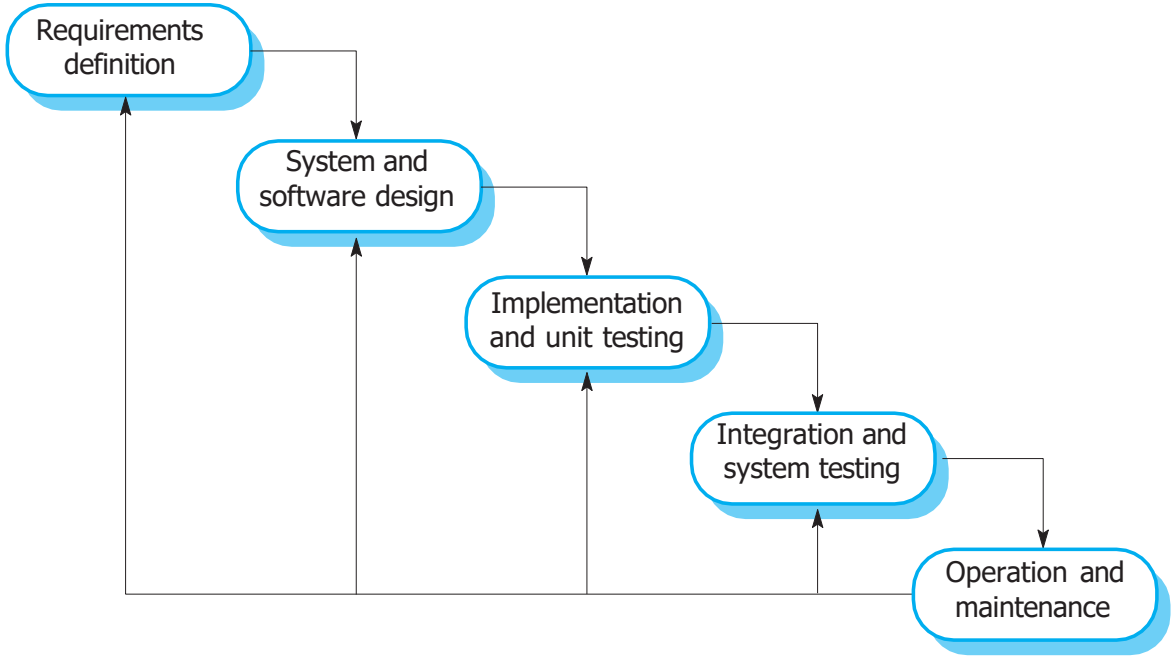
- In practice, most practical processes include elements of both plan-driven and agile approaches
- There are no right or wrong software processes

Software Process Models

- The waterfall model
  - ☞ Plan-driven model. Separate and distinct phases of specification and development
- Incremental development
  - ☞ Specification, development, and validation are done together. It can be plan-driven or agile.
- Integration and configuration
  - ☞ The system is built using existing configurable components. It can be plan-driven or agile.
- In practice, Most large systems use a combination of elements from these models.

Waterfall Model Phases

- There are separate identified phases in the waterfall model:

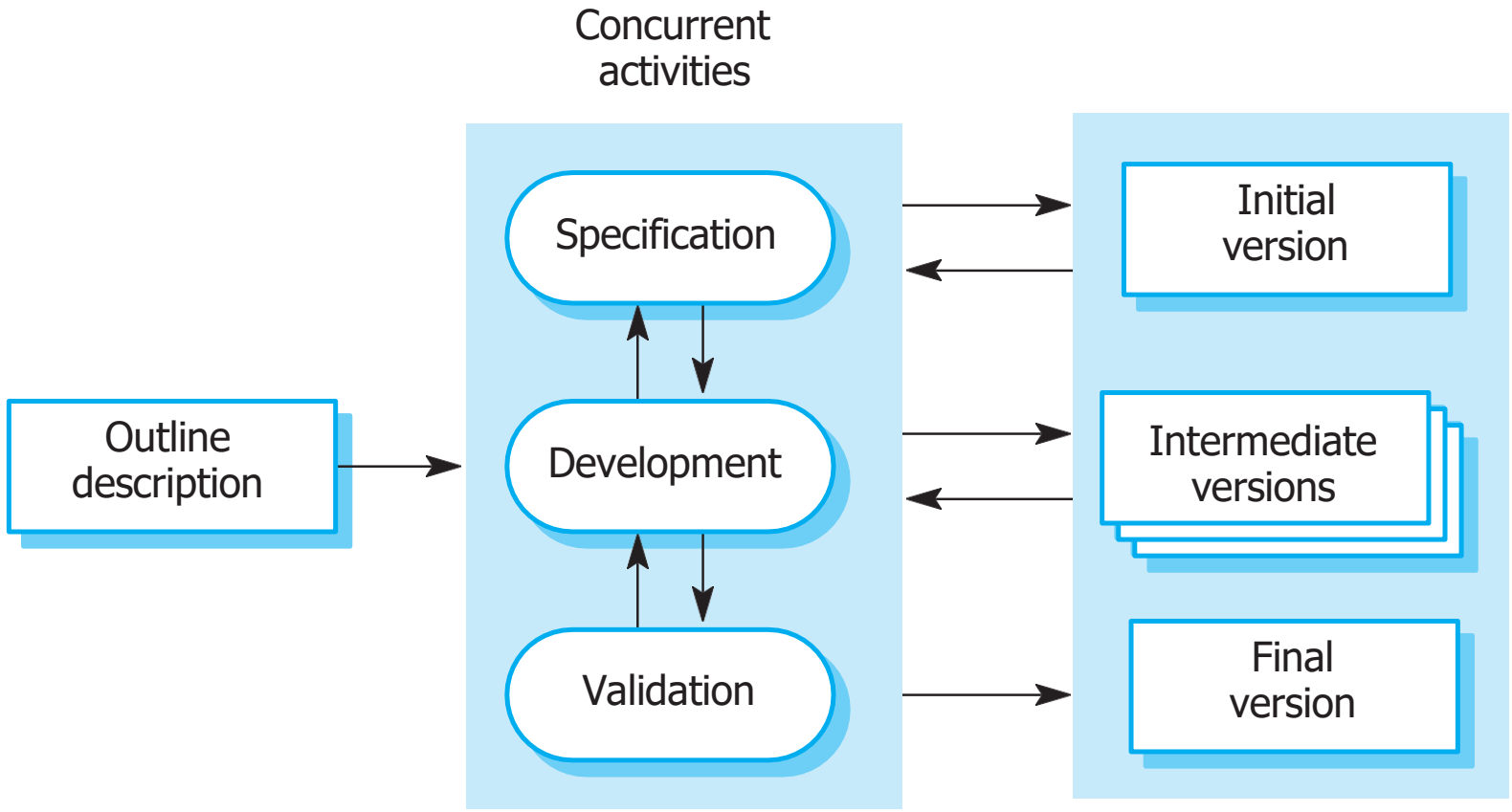


- The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway. In principle, a phase has to be complete before moving onto the next phase.

Waterfall Model Problems

- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements
  - Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process
  - Few business systems have stable requirements
- The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites
  - In those circumstances, the plan-driven nature of the waterfall model helps coordinate the work

Incremental Development



Incremental Development Benefits

- The cost of accommodating changing customer requirements is reduced
  - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model
- It is easier to get customer feedback on the development work that has been done
  - Customers can comment on demonstrations of the software and see how much has been implemented
- More rapid delivery and deployment of useful software to the customer is possible
  - Customers are able to use and gain value from the software earlier than is possible with a waterfall process

Incremental Development Problems

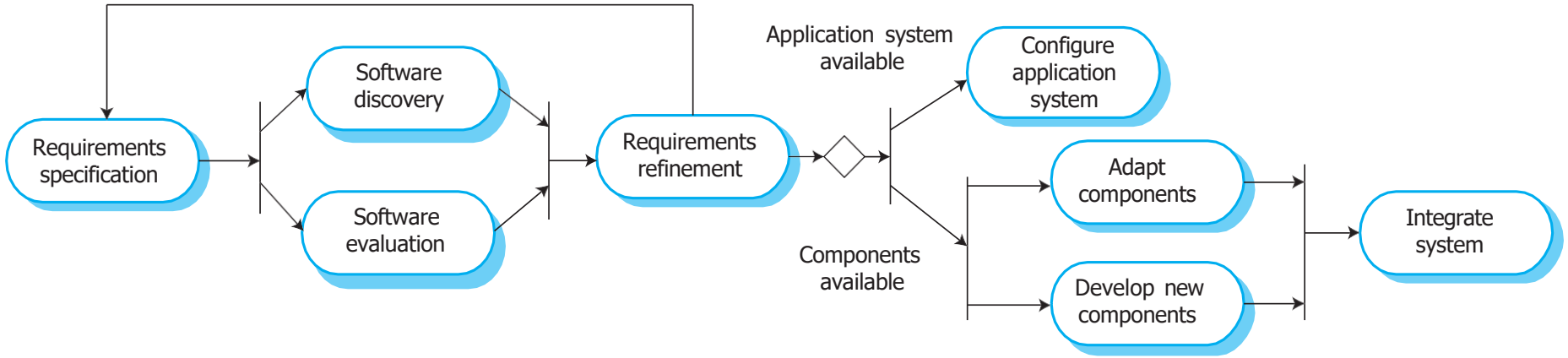
- The process is not visible
  - Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system
- System structure tends to degrade as new increments are added
  - Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly

Integration and Configuration for current practice

- Based on software reuse where systems are integrated from existing components or application systems (sometimes called COTS -Commercial-off-the-shelf systems)
- Reused elements may be configured to adapt their behaviour and functionality to a user's requirements
- Reuse is now the standard approach for building many types of business system

Reuse-oriented Software Engineering

- Requirements specification
- Software discovery and evaluation
- Requirements refinement
- Application system configuration
- Component adaptation and integration



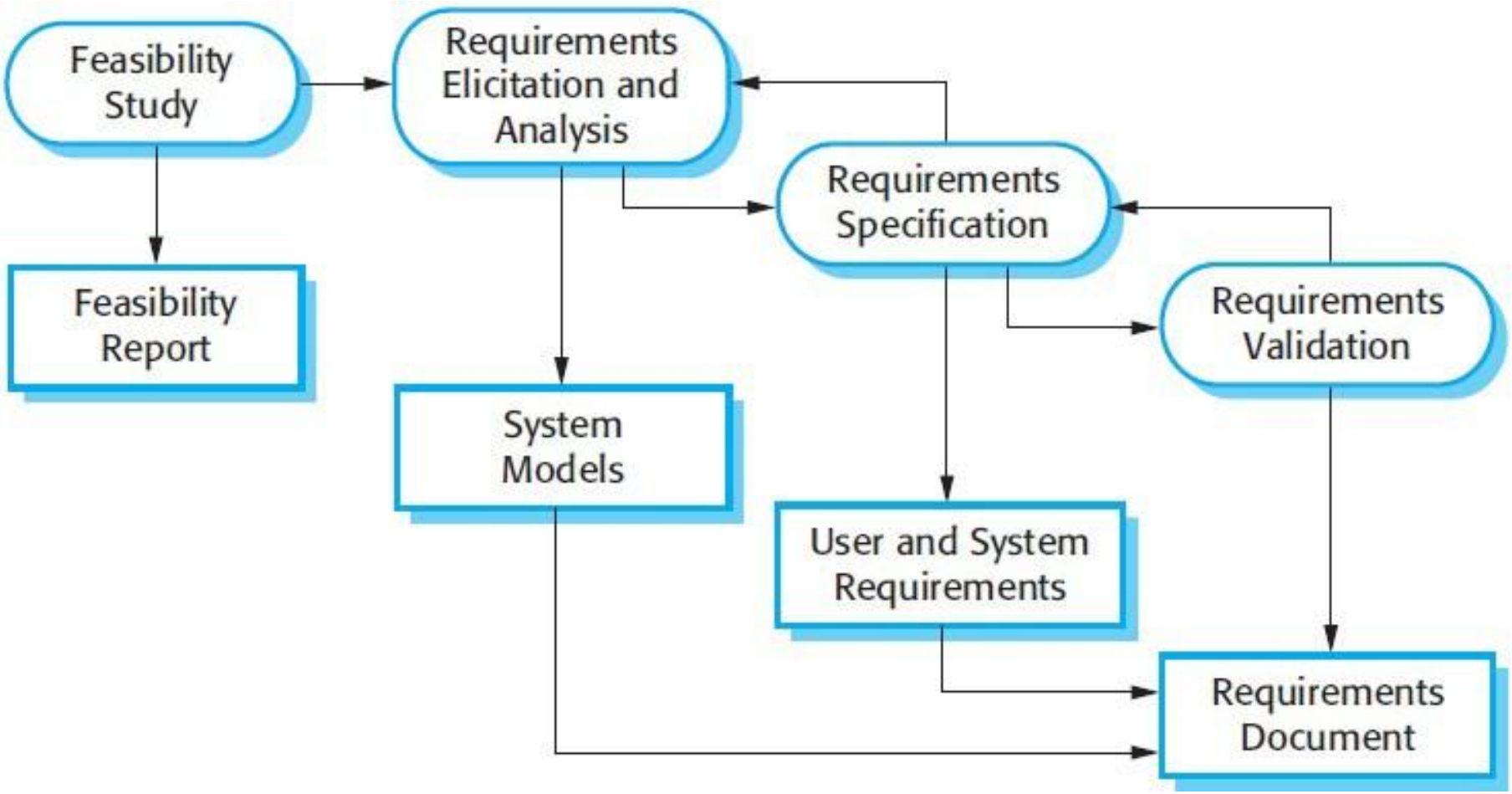
Reuse-oriented Software Engineering: Pros and Cons

- Reduced costs and risks as less software is developed from scratch
- Faster delivery and deployment of system
- But requirements compromises are inevitable so system may not meet real needs of users
- Loss of control over evolution of reused system elements

Software Specification

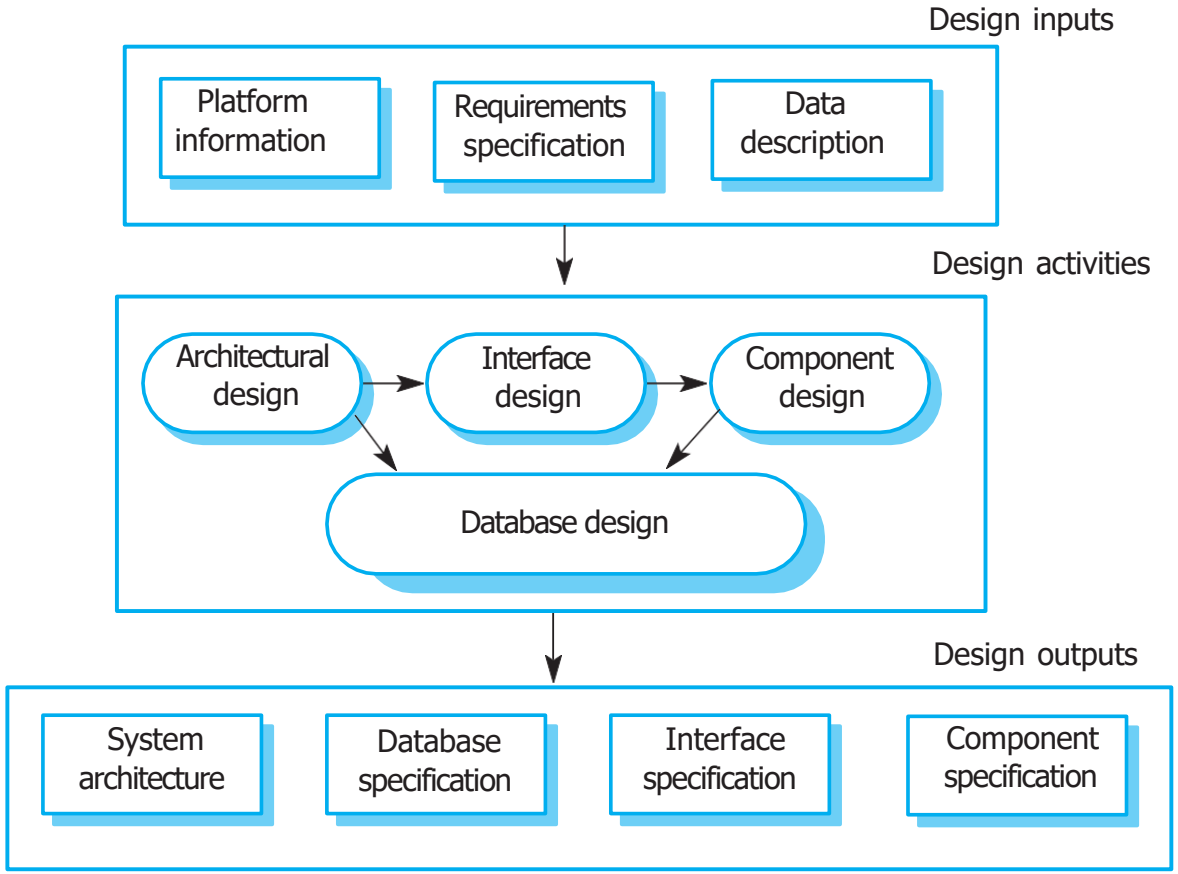
- The process of establishing what services are required and the constraints on the system's operation and development
- Requirements engineering process
  - Requirements elicitation and analysis
    - What do the system stakeholders require or expect from the system?
  - Requirements specification
    - Defining the requirements in detail
  - Requirements validation
    - Checking the validity of the requirements

The Requirements Engineering Process



A General Model of the Design Process

- *Architectural design*, where you identify the overall structure of the system, the principal components (subsystems or modules), their relationships and how they are distributed
- *Component design*, where you take each component and design how it will operate, with the specific design left to the programmer
- *Interface design*, where you define the interfaces between system components
- *Database design*, where you design the system data structures and how these are to be represented in a database



System Implementation

the process of turning software designs into a working system that users can interact with. It typically includes **programming**, **configuring systems**, and **debugging**.

- The software is implemented either by developing a program or programs or by configuring an application system
- Programming is an individual activity with no standard process
- Debugging is the activity of finding program faults and correcting these faults

Software Validation

Its purpose is to ensure the software meets its intended goals **accurately and reliably** before it's delivered to users.

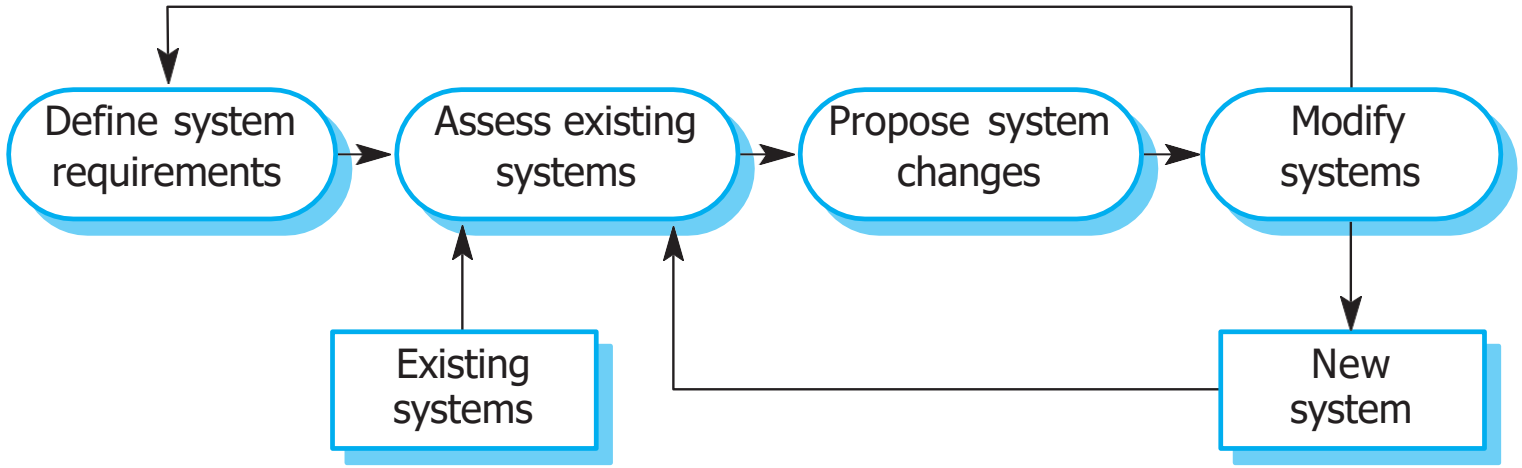
- Verification and validation (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer
- Involves checking and review processes and system testing
- System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system
- Testing is the most commonly used V & V activity

Testing Stages

- Component testing
  - Individual components are tested independently
  - Components may be functions or objects or coherent groupings of these entities
- System testing
  - Testing of the system as a whole. Testing of emergent properties is particularly important
- Acceptance testing
  - Testing with customer data instead of simulated data to check that the system meets the customer's needs

Software Evolution

- Software is inherently flexible and can change
- As requirements change through changing business circumstances, the software that supports the business must also evolve and change
- Although there has been a clear separation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new



Coping with Changing Requirements

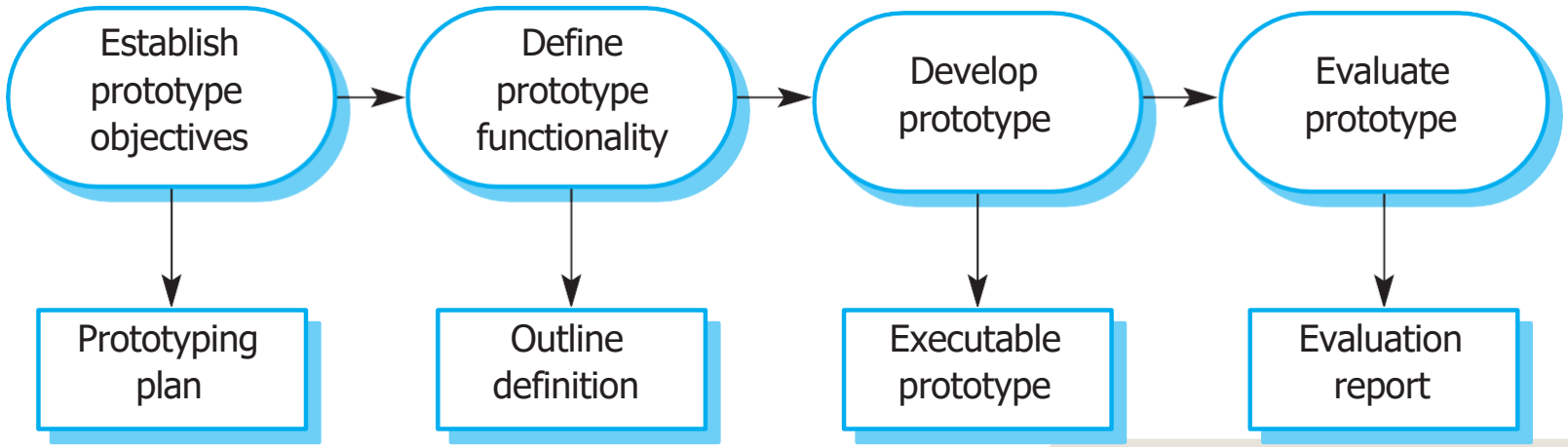
- System prototyping, where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of design decisions. This approach supports change anticipation.
- Incremental delivery, where system increments are delivered to the customer for comment and experimentation. This supports both change avoidance and change tolerance.

Software Prototyping

- A prototype is an initial version of a system used to demonstrate concepts and try out design options
- A prototype can be used in:
  - The requirements engineering process to help with requirements elicitation and validation
  - In design processes to explore options and develop a UI design
  - In the testing process to run back-to-back tests (involving two or more components of different versions with the same series of data at different conditions)

Prototype Development

- May be based on rapid prototyping languages or tools
- May involve leaving out functionality:
  - Prototype should focus on areas of the product that are not well-understood
  - Error checking and recovery may not be included in the prototype
  - Focus on functional rather than non-functional requirements such as reliability and security



Incremental Delivery

- Rather than deliver the system as a single delivery, the development and delivery is broken down into increments with each increment delivering part of the required functionality
- User requirements are prioritised and the highest priority requirements are included in early increments
- Once the development of an increment is started, the requirements are frozen though requirements for later increments can continue to evolve

Incremental Development and Delivery

This approach breaks down the process of building and releasing software into **smaller, manageable chunks**, allowing for **faster feedback**, **reduced risk**, and **better adaptability**. It's common in **Agile methodologies** and supports **continuous improvement**.

- Incremental development
  - Develop the system in increments and evaluate each increment before proceeding to the development of the next increment
  - Normal approach used in agile methods
  - Evaluation done by user/customer proxy
- Incremental delivery
  - Deploy an increment for use by end-users
  - More realistic evaluation about practical use of software
  - Difficult to implement for replacement systems as increments have less functionality than the system being replaced

Incremental Delivery Advantages

- Customer value can be delivered with each increment so system functionality is available earlier
- Early increments act as a prototype to help elicit requirements for later increments
- Lower risk of overall project failure
- The highest priority system services tend to receive the most testing

Incremental Delivery Problems

- Most systems require a set of basic facilities that are used by different parts of the system
  - As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments
- The essence of iterative processes is that the specification is developed in conjunction with the software
  - However, this conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract

Process Improvement

the **ongoing effort to refine and optimise the way software is developed**. It helps organisations deliver better software faster and more cost-effectively.

- Many software companies have turned to software process improvement as a way of enhancing the quality of their software, reducing costs or accelerating their development processes
- Process improvement means understanding existing processes and changing these processes to increase product quality and/or reduce costs and development time

Approaches to improvement

- *Process maturity approach* - focuses on improving process and project management and introducing good software engineering practice
  - The level of process maturity reflects the extent to which good technical and management practice has been adopted in organizational software development processes
- *Agile approaches aim to simplify processes and increase flexibility, with the goal of delivering value faster and responding to change more effectively.*
- The primary characteristics of agile methods are rapid delivery of functionality and responsiveness to changing customer requirements

Process Metrics

- Time taken for process activities to be completed
  - E.g. Calendar time or effort to complete an activity or process
- Resources required for processes or activities
  - E.g. Total effort in person-days
- Number of occurrences of a particular event
  - E.g. Number of defects discovered

Process Measurement

- Wherever possible, quantitative process data should be collected
  - However, where organisations do not have clearly defined process standards this is very difficult as you don't know what to measure. A process may have to be defined before any measurement is possible
- Process measurements should be used to assess process improvements
  - But this does not mean that measurements should drive the improvements. The improvement driver should be the organizational objectives

Class activity

In connection with the discussion slides, please download the class activity and complete it.

Question: compared to your previous programming assignments, what differences has software process made to your software development task? (related to Assessment 3)